

Experiment 2

Aim: Write a program to implement Group Communication

Theory:

Group Communication in Distributed Systems

Group communication in distributed systems refers to the process where multiple nodes or entities communicate with each other as a group.

- Instead of sending messages to individual recipients, group communication allows a sender to transmit information to all members of a group simultaneously.
- This method is essential for coordinating actions, sharing data, and ensuring that all participants in the system are informed and synchronized. It's particularly useful in scenarios like collaborative applications and real-time updates.

Importance of Group Communication in Distributed Systems

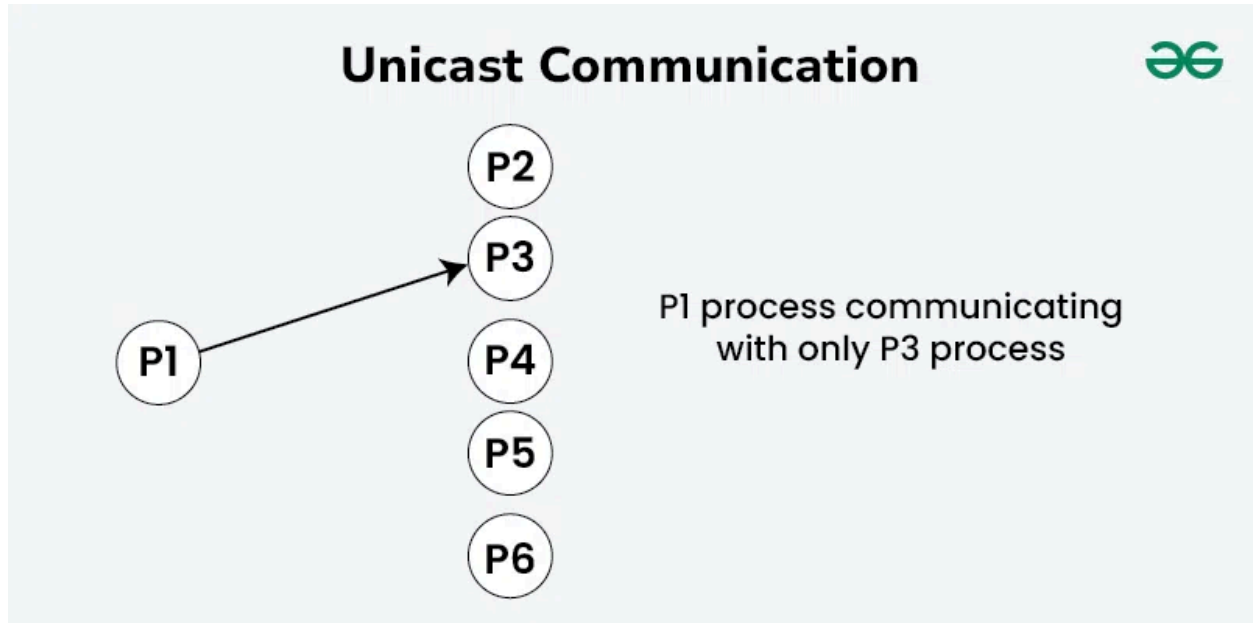
- Multiple nodes must collaborate and synchronize their actions. Group communication helps them exchange information and stay updated.
- Different nodes can create data that needs to be shared. Group communication helps quickly send this information to everyone involved, reducing delays and keeping data consistent.
- Group communication protocols enhance reliability by allowing messages to be replicated or acknowledged across multiple nodes. This ensures robust communication, even during failures or network issues.
- As distributed systems expand, effective scaling is crucial. Group communication mechanisms can manage more nodes and messages without sacrificing performance, keeping the system efficient and responsive.

Types of Group Communication in a Distributed System

1. Unicast Communication

Unicast communication is the point-to-point transmission of data between two nodes in a network. In the context of distributed systems:

- Unicast is when a sender sends a message to a specific recipient, using their unique network address.

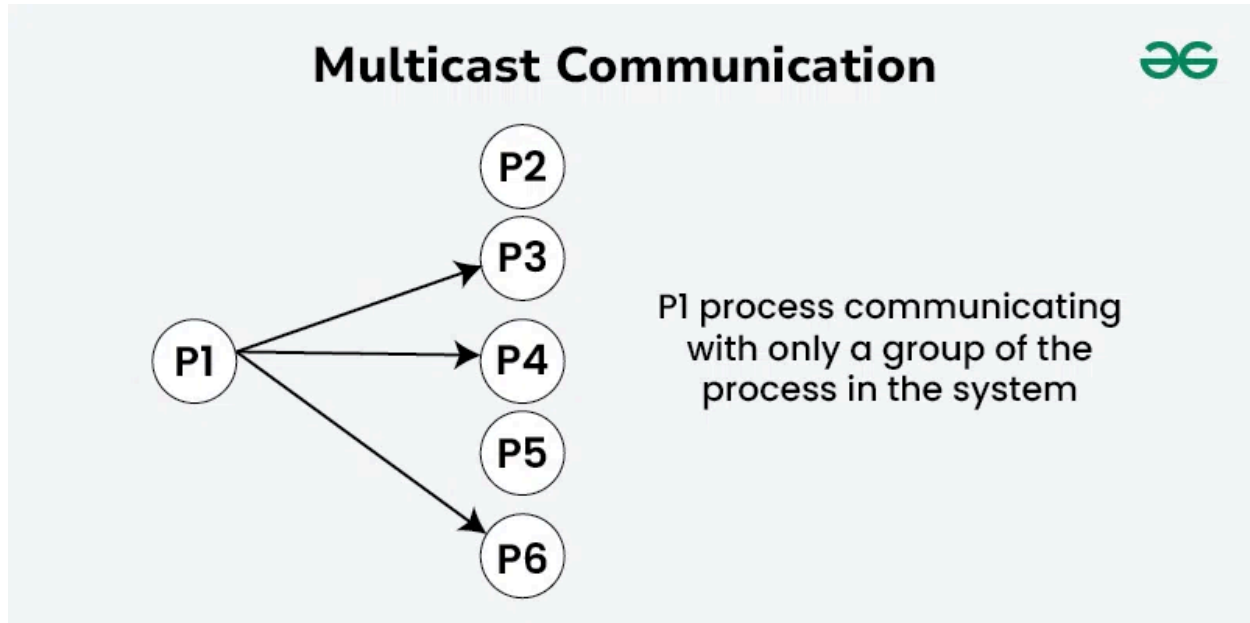


- Each message targets one recipient, creating a direct connection between the sender and the receiver.
- You commonly see unicast in client-server setups, where a client makes requests and receives responses, as well as in direct connections between peers.
- This method makes good use of network resources, is easy to implement, and keeps latency low because messages go straight to the right person.
- Unicast isn't efficient for sending messages to many recipients at once, as it requires separate messages for each one, leading to more work.

2. Multicast Communication

Multicast communication involves sending a single message from one sender to multiple receivers simultaneously within a network. It is particularly useful in distributed systems where broadcasting information to a group of nodes is necessary:

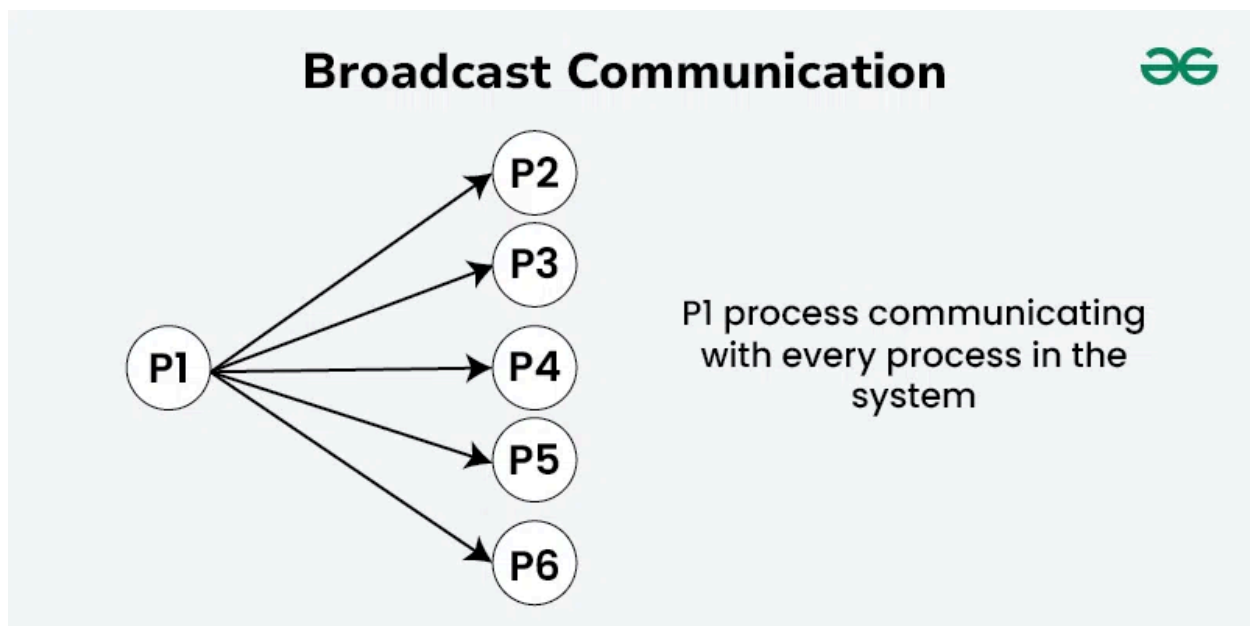
- Multicast lets a sender share a message with a specific group of people who want it.
- This way, the sender can reach many people at once, which is more efficient than sending separate messages.
- This approach is often used to send updates to subscribers or in collaborative applications where real-time sharing of changes is needed.
- By sending data just once to a group, multicast saves bandwidth, simplifies communication, and can easily handle a larger number of recipients.



- Managing group membership is necessary to ensure reliable message delivery, and multicast can run into issues if there are network problems that affect everyone in the group.

3. Broadcast Communication

Broadcast communication involves sending a message from one sender to all nodes in the network, ensuring that every node receives the message:



- Broadcast is when a sender sends a message to every node in the network without targeting specific recipients.
- Messages are delivered to all nodes at once using a special address designed for this purpose.
- It's often used for network management tasks, like sending status updates, or for emergency alerts that need to reach everyone quickly.
- Broadcast ensures that every node receives the message without needing to specify who the recipients are, making it efficient for sharing information widely.
- It can cause network congestion in larger networks and raises security concerns since anyone on the network can access the broadcast message, which might lead to unauthorized access.

Code:

1. Server.java

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) throws IOException {
        int PORT = 2004;
        ServerSocket serverSocket = new ServerSocket(PORT);
        System.out.println("Server started on port " + PORT);

        while (true) {
            Socket client = serverSocket.accept();
            System.out.println("Client connected: " +
client.getInetAddress());
            new ClientHandler(client).start();
        }
    }
}

class ClientHandler extends Thread {
    private final Socket client;

    public ClientHandler(Socket client) {
        this.client = client;
    }
}
```

```

    }

    public void run() {
        try (BufferedReader input = new BufferedReader(new
InputStreamReader(client.getInputStream()));
            PrintWriter output = new
PrintWriter(client.getOutputStream(), true)) {

            output.println("Server is working:");
            String message;
            while ((message = input.readLine()) != null) {
                System.out.println("Received: " + message);
                output.println("Server message: " + message);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

2. Client.java

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {
    public static void main(String[] args) throws IOException {
        String HOST = "127.0.0.1";
        int PORT = 2004;

        Socket socket = new Socket(HOST, PORT);
        BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        PrintWriter output = new PrintWriter(socket.getOutputStream(),
true);
        Scanner scanner = new Scanner(System.in);

        System.out.println(input.readLine()); // Server welcome message
    }
}

```

```

while (true) {
    System.out.print("Enter message: ");
    String message = scanner.nextLine();
    output.println(message);
    System.out.println(input.readLine()); // Server response
}
}
}

```

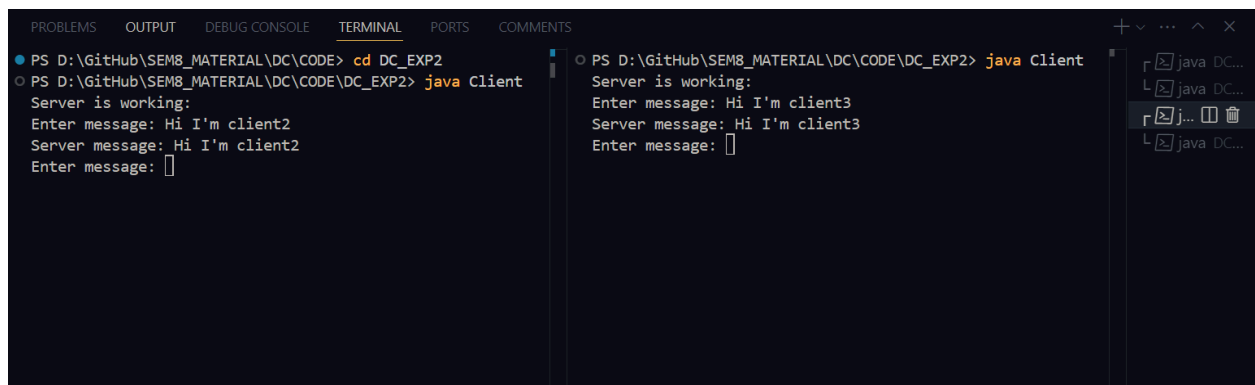
Output:



```

PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2> javac Server.java Client.java
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2> java Server
Server started on port 2004
Client connected: /127.0.0.1
Received: Hey there!
Client connected: /127.0.0.1
Received: Hi I'm client2
Client connected: /127.0.0.1
Received: Hi I'm client3
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2> java Client
Server is working:
Enter message: Hey there!
Server message: Hey there!
Enter message:

```



```

PS D:\Github\SEM8_MATERIAL\DC\CODE> cd DC_EXP2
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2> java Client
Server is working:
Enter message: Hi I'm client2
Server message: Hi I'm client2
Enter message:
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP2> java Client
Server is working:
Enter message: Hi I'm client3
Server message: Hi I'm client3
Enter message:

```